

Node.js

Building Scalable and Efficient Web Applications

Here is where your presentation begins



Node.js

HTTP & HTTP METHODS

- Sandeep Jaiswal (Technical lead @ Wipro)



1. HTTP & HTTP Methods

01

Introduction

02

Scenario on HTTP

03

HTTP Methods

04

HTTP Response Structure

05

HTTP status code

06

HTTP Response Body

07

HTTP vs HTTPS



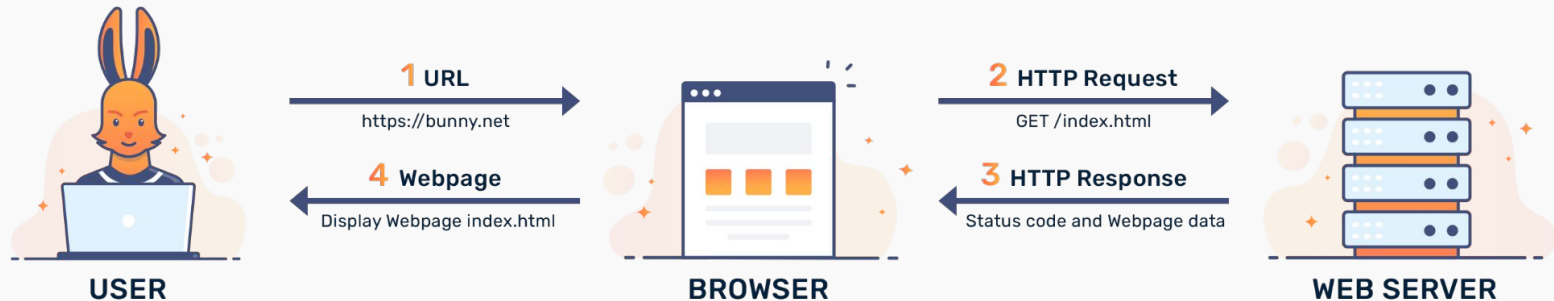
01

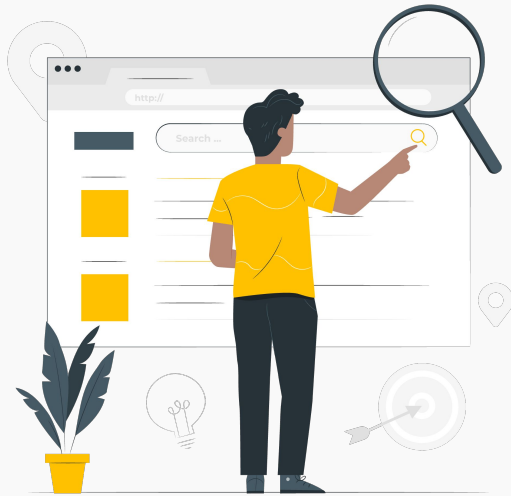
Introduction

Introduction

- HTTP stands for **HyperText Transfer Protocol**.
- It is a protocol used to access the data on the World Wide Web (www).
- The HTTP protocol can be used to transfer the data in the form of plain text, hypertext, audio, video, and so on.
- An HTTP request is the way internet communications platforms such as web browsers ask for the information they need to load a website.

COMMUNICATION PROCESS ACCORDING TO HTTP



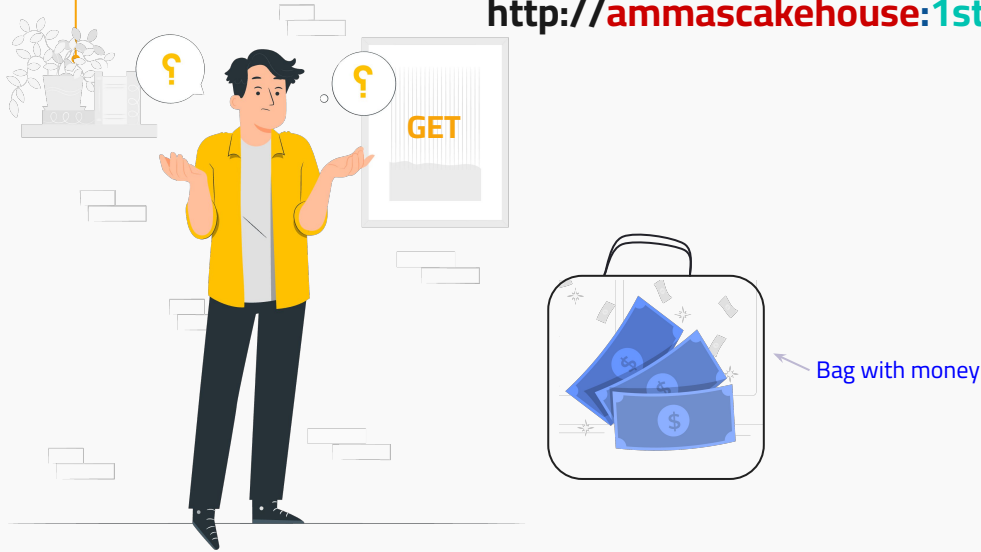


02

Scenario on HTTP protocol structure

Scenario explain HTTP call Structure

`http://ammascakehouse:1st main road/cakelist/redvelvet`



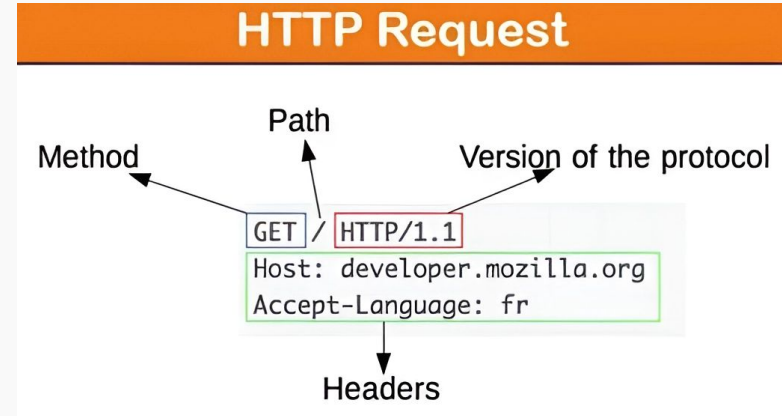
Ammascakehouse	⇒	Name of server or shop
: 1st main road	⇒	Port or the area where shop located
GET	⇒	HTTP method or asking to get cake.
Bag with money	⇒	Header information or securely giving money

HTTP Protocol Structure

Each HTTP request made across the Internet carries with it a series of encoded data that carries different types of information. A typical HTTP request contains:

1. A URL
2. An HTTP method
3. HTTP request headers
4. HTTP body (optional).
5. Status code

The url has following structure and meaning:-





03

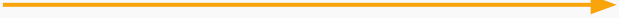
HTTP Methods in detail

HTTP Methods - 1 (CRUD)

HTTP defines several methods to perform different actions on web resources:

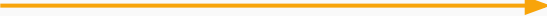
1. GET Method ⇒ To retrieve data from a server, No request body (parameters are passed via URL or query strings).

```
GET /api/customers/123
```



2. POST Method ⇒ To send data to the server to create a new resource or submit data (e.g., form submissions). Data is sent in the request body.

```
POST /api/customers/register
```



```
{  
  "name" : "John",  
  "email" : "john@gmail.com",  
  "password" : "asdf1234"  
}
```

HTTP Methods - 2

1. PUT Method ⇒ To update an existing resource or create a resource if it doesn't exist. Sends data in the request body.

PUT /api/customers/123

```
{  
  "name" : "John",  
  "email" : "john@gmail.com",  
  "password" : "john@admin123"  
}
```

2. DELETE Method ⇒ To delete a resource from the server. No request body is usually needed.

DELETE /api/customers/123



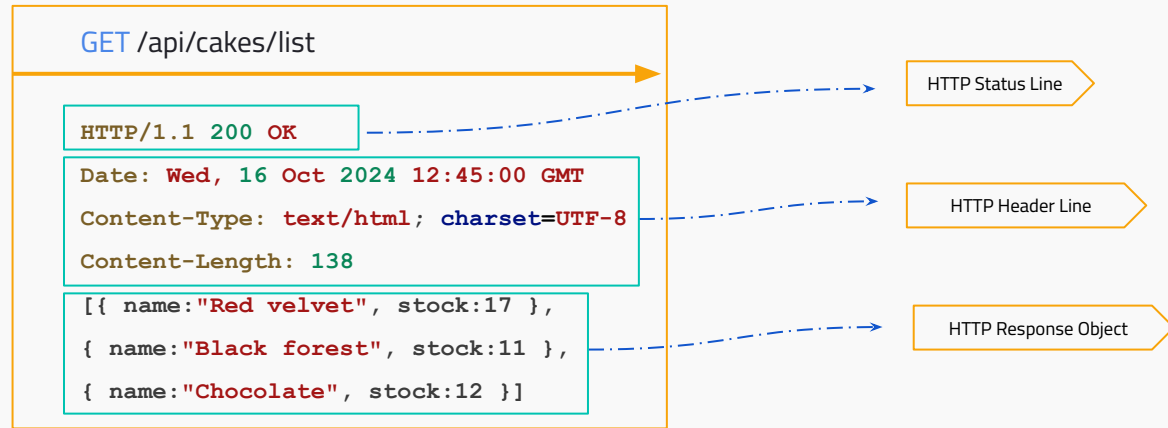
04

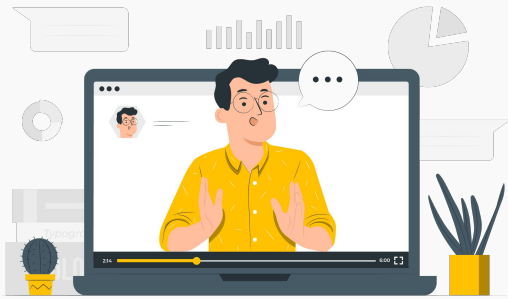
HTTP Response Structure

HTTP Response Structure

An HTTP response is a structured communication from the server that provides the client with the requested data or an indication of the request's outcome.

The combination of the status line, headers, and body ensures that the client knows how to interpret and use the response.





05

HTTP Status code

HTTP Status Code

After a request, the server responds with a status code that indicates the result of the operation. These are categorized into five main classes:

1xx (Informational): The request was received, and the server is continuing the process (e.g., **100 Continue**).

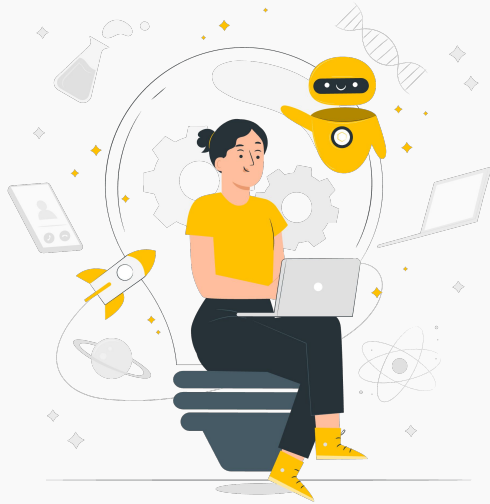
2xx (Success): The request was successfully received and processed (e.g., **200 OK**).

3xx (Redirection): The client must take additional action to complete the request (e.g., **301 Moved Permanently**).

4xx (Client Errors): The request contains bad syntax or cannot be fulfilled (e.g., **404 Not Found**).

5xx (Server Errors): The server failed to fulfill a valid request (e.g., **500 Internal Server Error**).

200 OK	201 CREATED	301 MOVED	401 UNAUTHORIZED
403 FORBIDDEN	404 NOT FOUND	405 METHOD NOT ALLOWED	413 PAYLOAD TOO LARGE
500 INTERNAL SERVER ERROR	502 BAD GATEWAY	503 SERVER UNREACHABLE	



06

HTTP Response Body

HTTP Response body

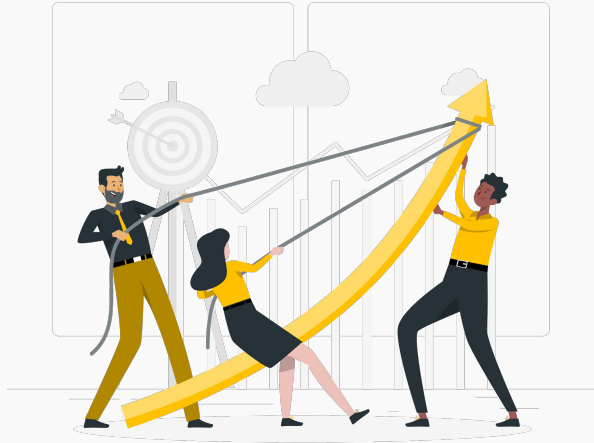
It contains the actual data that the server sends back to the client after processing the request.

The format of the body depends on the **Content-Type** header.

Note:- The body is typically sent only in successful (e.g., **200 OK**) or informational (e.g., **201 Created**)

Common content types include:

- **text/html**: HTML content
- **text/plain**: Plain text
- **application/json**: JSON data
- **image/jpeg**: JPEG image
- **application/pdf**: PDF document



07

HTTP vs HTTPS

HTTP vs HTTPS

HTTP (HyperText Transfer Protocol): A protocol used for transmitting data between a client (browser) and a server in plain text.

HTTPS (HyperText Transfer Protocol Secure): A secure version of HTTP. It uses encryption (via TLS/SSL) to ensure that data sent between the client and server is encrypted, providing privacy, security.

Feature	HTTP	HTTPS
Encryption	No encryption is used.	Encrypts data to secure communication
Port	Default port is 80 .	Default port is 443
Performance	Slightly faster as it doesn't require the overhead of encryption and decryption.	Slightly slower due to the overhead of encrypting and decrypting data

Summary on HTTP

What is HTTP?

- HyperText Transfer Protocol, used for communication between client (browser) and server

Structure of HTTP Response

- Status Line, Response Headers, Response Body

HTTP Status Codes

- 1xx: Informational (e.g., 100 Continue), 2xx: Success (e.g., 200 OK), 3xx: Redirection (e.g., 301 Moved Permanently), 4xx: Client errors (e.g., 404 Not Found), 5xx: Server errors (e.g., 500 Internal Server Error)

Common Response Formats

- HTML, JSON/XML, Plain Text, Binary Data

Important HTTP Response Headers

- Content-Type, Content-Length, Set-Cookie, Cache-Control

Node.js

GLOBAL OBJECTS

- Sandeep Jaiswal (Technical lead @ Wipro)



2. GLOBAL OBJECTS & MODULE

01

Introduction

02

Global Objects

03

Default Modules



01

Introduction

Introduction

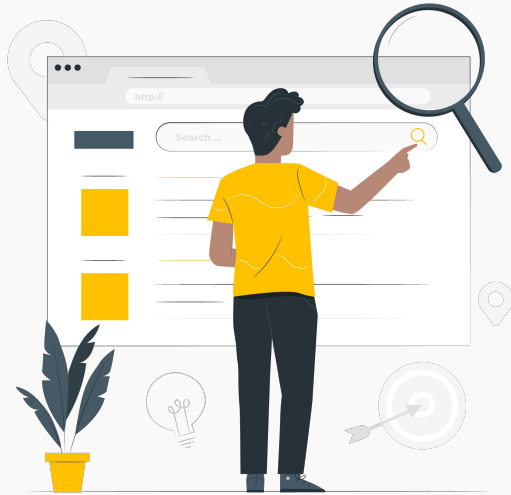
- Node.js provides a rich set of **global objects** and **default modules** that empower us to build scalable and efficient applications.
- Default modules, known as **core modules**, are modules that come bundled with Node.js.

List of Global Objects:

global	process	console	__dirname	__filename	Buffer
setTimeout(), clearTimeout()		setInterval(), clearInterval()		setImmediate(), clearImmediate()	

List of Global Modules:

fs	path	http	https	url	util	events
stream	os	crypto	querystring	zlib	dns	net



02

Global Objects

Global Objects in detail

1. global

The `global` object is the global namespace in Node.js.

It provides access to various built-in functions, properties, and objects that are available everywhere in your Node.js application.

```
/* index.js */  
// Declare a global variable  
global.appName = 'apple';  
  
console.log(global.appName);  
// Outputs: MyNodeApp
```

```
/* data.js */  
  
require("./index.js");  
  
console.log(global.appName);  
// Outputs: MyNodeApp
```

Process Objects in detail



2. process

It provides information about the currently running Node.js process and offers control over it.

An object representing the Node.js process itself, providing information about the environment, memory usage, and more.

List of properties of process Object:-

`argv` `env` `version` `platform` `arch` `stdin` `stdout` `buildConfig`

List of Methods of process Object:-

`exit()` `kill()` `nextTick()`

List of process object Properties in detail



1. argv

An array containing the command-line arguments passed when the Node.js process was launched.

```
console.log(process.argv);
```

```
// If run with `node app.js arg1 arg2`, it will output:
```

```
// ['/path/to/node', '/path/to/app.js', 'arg1', 'arg2']
```

2. env

An object containing the user environment variables at the time the process was started.

```
console.log(process.env.NODE_ENV); // Outputs the current environment (e.g., 'development')
```

List of process object Properties in detail



3. version

A string that contains the Node.js version.

```
console.log(process.version); // 'v22.7.0'
```

4. platform

A string identifying the operating system platform the Node.js process is running on.

```
console.log(process.platform); // Outputs: 'darwin', 'win32', 'linux', etc.
```

5. arch

A string identifying the processor architecture for which the Node.js binary was compiled.

```
console.log(process.arch); // Outputs: 'x64', 'arm', 'ia32', etc.
```

Console Object in detail

3. console

The `console` object is another globally available object .

It provides simple methods to print messages to the standard output (`stdout`) or standard error (`stderr`).

```
console.log('Logging information');    // Outputs a log message
console.error('Error:', new Error('Something went wrong')); //Outputs:Error:Something went wrong
```

List of Console Methods:

<code>log()</code>	<code>error()</code>	<code>warn()</code>	<code>info()</code>	<code>debug()</code>	<code>clear()</code>
<code>count()</code>	<code>table()</code>	<code>time()</code>	<code>timeEnd()</code>		

List of console object methods in detail



1. log()

Outputs a message to the standard output (typically the terminal).

It can accept multiple arguments and will automatically concatenate them into a string with spaces between them.

```
console.log('Hello, World!'); // Outputs: Hello, World!
console.log('Number:', 42);   // Outputs: Number: 42
console.log('Multiple', 'arguments', 'are', 'concatenated'); // Outputs: Multiple arguments are concatenated
```

2. error()

Outputs an error message to the standard error (`stderr`).

```
console.error('This is an error message'); // Outputs: This is an error message
console.error('Error:', new Error('Something went wrong')); // Outputs: Error: Something went wrong
```

List of console object methods in detail



3. warn()

Outputs a warning message to the standard error (`stderr`).

```
console.warn('This is a warning'); // Outputs: This is a warning
console.warn('Warning:', 'Low disk space'); // Outputs: Warning: Low disk space
```

2. info()

An alias for `console.log()`. It outputs an informational message to the standard output.

```
console.info("Info:", "Process started"); // Outputs: Info: Process started
```


List of console object methods in detail



3. debug()

An alias for `console.log()` but used specifically for debugging purposes.

```
console.debug('Debug:', { user: 'John Doe', age: 30 });  
// Outputs: Debug:{ user: 'John Doe', age: 30 }
```

2. clear()

Clears the console, if possible. In some environments, it may not work.

```
console.log("This message will be cleared");  
setTimeout(() => {  
    console.clear(); // Clears the console  
}, 2000);
```

List of console object methods in detail



3. table()

Displays tabular data as a table. The input can be an array or an object. Each element in the array (or object) will be a row in the table.

```
const people = [  
  { name: 'John', age: 30 },  
  { name: 'Jane', age: 28 },  
];  
console.table(people); // Displays the array as a table
```

3. time() & timeEnd()

Starts a timer with a specific label. The time elapsed until `console.timeEnd()` is called will be printed in milliseconds.

```
console.time('Timer');  
console.log("This console message");  
console.timeEnd('Timer'); // Outputs: Timer: <elapsed time>ms
```

__dirname & __filename properties

4. __dirname

__dirname is a global variable that contains the directory name of the current module file.

This is especially useful when working with file paths

```
console.log(__dirname);  
// Output :- /Users/sandeepjaiswal/Downloads/test-api
```

5. __filename

__filename is a global variable that contains the full path and filename of the current module file.

This is useful for logging or debugging purposes to see where the current script is located.

```
console.log(__filename);  
// Output :- /Users/sandeepjaiswal/Downloads/test-api/index.js
```

setTimeout method in detail

6. Timer :- setTimeout()

The `setTimeout()` function in JavaScript (including Node.js) is used to execute a function or a specified piece of code **after a set delay** (in milliseconds).

It is part of the asynchronous programming capabilities of JavaScript

Syntax:- `setTimeout(function, delay, arg1, arg2, ...)`

function: The function or block of code to execute after the delay.

delay: The time, in milliseconds, after which the function should be executed. **1000** milliseconds equals **1** second.

arg1, arg2, ...: Optional arguments passed to the function when it is called.

```
setTimeout(() => {  
    console.log('This message will be logged after 2 seconds');  
}, 2000); // 2000 milliseconds = 2 seconds
```

setInterval method in detail

7. Timer :- setInterval()

The `setInterval()` is a JavaScript function used to repeatedly call a function or execute a block of code at **specified time intervals** (in milliseconds)

It is part of the asynchronous programming capabilities of JavaScript

Syntax:- `setInterval(function, delay, arg1, arg2, ...)`

function: The function or block of code that you want to run repeatedly.

delay: The time, in milliseconds, after which the function should be executed. **1000** milliseconds equals **1** second.

arg1, arg2, ...: Optional arguments passed to the function when it is called.

```
setInterval(() => {  
    console.log("This message will repeat every 2 seconds");  
}, 2000); // 2000 milliseconds = 2 seconds
```

setImmediate method in detail

8. Timer :- setImmediate()

In Node.js, `setImmediate()` is a function that schedules a callback function to be executed **immediately after the current event loop cycle finishes**.

It's similar to `setTimeout()` with a 0 millisecond delay, but `setImmediate()` is more efficient

Syntax:- `setImmediate(callback, [arg1, arg2, ... ])`

callback: The function you want to execute after the current event loop finishes.

arg1, arg2, ...: Optional arguments that will be passed to the callback function when it is invoked.

```
console.log('Start');  
setImmediate(() => {  
    console.log('This runs after the current event loop');  
});  
console.log('End');
```

Summary on Global Objects

Node.js has several **global objects** that are available in all modules without the need to `require()` them.

global

- Equivalent to `window` in browsers, it is the global scope object in Node.js.

process

- Provides information and control over the current Node.js process

Common properties and methods include:

- `process.env`: Access environment variables.
- `process.argv`: Command-line arguments passed to the script.
- `process.exit()`: Exit the current process.

console

- it provides methods to output messages to the terminal.

Timer - `setTimeout`, `setInterval`, `setImmediate`

- Schedules a function to execute **based on time delay/ time interval or immediate**

Node.js

DEFAULT MODULES

- Sandeep Jaiswal (Technical lead @ Wipro)



Introduction



Node.js supports several types of modules, each serving different purposes and following various conventions.

1. Builtin Modules:

Node.js comes with a set of built-in modules that you can use without installing any additional packages

fs	path	http	https	url	util	events
stream	os	crypto	querystring	zlib	dns	net

2. User-defined Modules

You can create your own modules by organizing your code into separate JavaScript files.

```
// index.js
const greet = (name) => `Hello, ${name}!`;
module.exports = { greet };
```

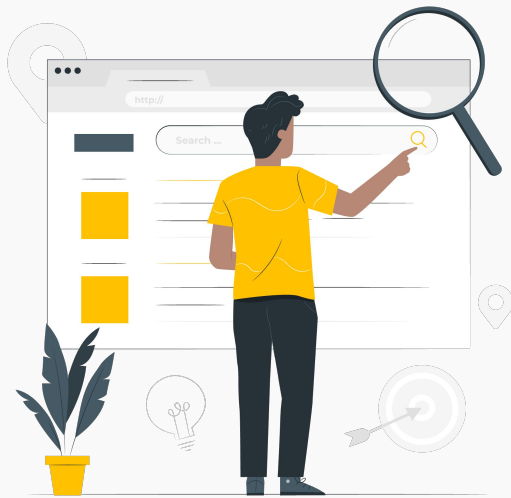
```
// data.js
const indexMod = require("./index");
console.log(indexMod.greet("Sandeep")); // Output: Hello, Sandeep!
```

3. Third Party Modules:

These are modules that are not included with Node.js but can be installed from the npm (Node Package Manager).

Popular Third-Party Modules: visit <https://www.npmjs.com/> for reference

- **express:** A web application framework for building web and mobile applications.
- **mongoose:** An ODM (Object Data Modeling) library for MongoDB and Node.js.



01

File System (fs)

fs module in detail

1. fs

The **fs** (File System) module in Node.js allows you to interact with the file system.

Its helps us to interact file in both way, synchronous and asynchronous way.

It provides a variety of methods to read, write, update, delete, and manipulate files and directories.

1. Loading the **fs** module:

```
const fs = require('fs');
```

The fs module is used as two way operation on file•– Asynchronous – Synchronous

Some of the common methods used:-

<code>readFile()</code>	<code>readFileSync()</code>	<code>writeFile()</code>	<code>writeFileSync()</code>
<code>appendFile()</code>	<code>appendFileSync()</code>	<code>unlink()</code>	<code>unlinkSync()</code>
<code>mkdir()</code>	<code>mkdirSync()</code>	<code>rmdir()</code>	<code>rmdirSync()</code>
<code>stat()</code>	<code>statSync()</code>	<code>isFile()</code>	<code>isDirectory()</code>
<code>createReadStream()</code>	<code>createWriteStream()</code>		

List of fs methods in detail

1. readFileSync()

The `fs.readFileSync()` method in Node.js is used to **synchronously read the contents of a file**. This method blocks the execution of code until the file reading operation is complete.

Syntax:-

```
fs.readFileSync(path, [options])
```

Parameters:

1. **path** (String or Buffer or URL or File descriptor):
 - You can provide either relative or absolute file paths.
2. **options** (Optional):
 - Can be either an object or a string. It determines the encoding and the mode used when reading the file.
 - **Encoding:** If not specified, it will return the raw buffer data. If specified as `'utf8'`, `'ascii'`, or other encodings, the method returns a string.

List of fs methods in detail

1. Reading file with Encoding

The `fs.readFileSync()` method reads the contents of a file in a **synchronous** manner and logs the content as a string (using 'utf8' encoding).

```
// index.js
const fs = require("fs");
```

```
// abc.txt
This is the content.
```

```
// Reading file with utf8 encoding
const data = fs.readFileSync("./abc.txt", "utf8");

console.log(data); // This is the content.
```

List of fs methods in detail

2. readFile()

The `fs.readFile()` method in Node.js is used to **asynchronously read the contents of a file**.

It is part of the Node.js `fs` (File System) module and follows the asynchronous programming model.

The function returns immediately, and the results are provided through a callback once the file reading is completed.

Syntax:- `fs.readFile(path, [options], callback)`

Parameters:

1. **path** (String or Buffer or URL or File descriptor):
 - You can provide either relative or absolute file paths.
2. **options** (Optional):
 - **Encoding:** If not specified, it will return the raw buffer data. If specified as `'utf8'`, `'ascii'`, or other encodings, the method returns a string.
3. **callback:**
 - The callback function is called after the file has been read. It takes two parameters:
 - **err:** Error object if there was an error reading the file (e.g., if the file doesn't exist).
 - **data:** The data read from the file, either as a string (if encoding is provided) or as a Buffer (if no encoding is specified).

List of fs methods in detail

Reading file with Encoding

In this example, we are reading the contents of a file asynchronously and providing a callback to handle the result.

```
// index.js
const fs = require("fs");
```

```
// abc.txt
This is the content.
```

```
// Read a file asynchronously with 'utf8' encoding
fs.readFile("./abc.txt", "utf8", (err, data) => {
  if (err) {
    console.error("Error reading file:", err);
    return;
  }
  console.log("File content:", data); // This is the content.
});
```

List of fs methods in detail



3. writeFileSync()

The `fs.writeFileSync()` method in Node.js is used to **synchronously write data to a file**.

Syntax:- `fs.writeFileSync(path, data, [options])`

Parameters:

1. **data** (String | Buffer | TypedArray | DataView | Object):
 - The content that you want to write to the file.
 - This can be a string, a Buffer (for binary data), or any other form of data that can be converted to a buffer.

This method does **not return a value**. It simply writes the data to the file and then allows the program to proceed. If there is an error, it throws an exception.

```
const fs = require('fs');                                // data.txt
// Write text to a file synchronously                      Hello World!
fs.writeFileSync('data.txt', 'Hello, World!');
console.log('File written successfully');
```


List of fs methods in detail

4. writeFile()

The `fs.writeFile()` method in Node.js is used to **asynchronously write data to a file**.

Syntax:- `fs.writeFile(path, data, [options], callback)`

```
const fs = require("fs");

// Write a string to a file asynchronously
fs.writeFile("list.txt", "Hello, World!", (err) => {
  if (err) {
    console.error("Error writing file:", err);
    return;
  }
  console.log("File written successfully");
});
```

// list.txt
Hello, World!

List of fs methods in detail

4. appendFile()

The `fs.appendFile()` method in Node.js is used to **asynchronously append data to a file**.

If the file does not exist, it will be created.

Syntax:- `fs.appendFile(path, data, [options], callback)`

```
const fs = require('fs');  
// Append a string to a file  
fs.appendFile('./list.txt', '\nHow are you \n', (err) => {  
  if (err) {  
    console.error('Error appending to file:', err);  
    return;  
  }  
  console.log('Data appended to file successfully');  
});
```

```
// list.txt  
Hello, World!  
How are you
```

List of fs methods in detail

4. `unlink()`

The `fs.unlink()` method in Node.js is used to **asynchronously delete a file or symbolic link**.

If the file does not exist, it will be created.

Syntax:- `fs.unlink(path, callback)`

```
const fs = require("fs");  
// Delete a file asynchronously  
fs.unlink("data.txt", (err) => {  
    if (err) {  
        console.error("Error deleting the file:", err);  
        return;  
    }  
    console.log("File deleted successfully");  
});
```

// OutPut
File deleted successfully

If `file.txt` exists, it will be deleted. If it doesn't exist, the `err` object will contain details about the error.

List of fs methods in detail

4. `mkdir()`

The `fs.mkdir()` method in Node.js is used to **asynchronously create a new directory**.

Syntax:- `fs.mkdir(path,[options], callback)`

```
const fs = require("fs");
```

```
// Create a new directory called 'newDir'
```

```
fs.mkdir("newDir", (err) => {
```

```
  if (err) {
```

```
    console.error("Error creating directory:", err);
```

```
    return;
```

```
  }
```

```
  console.log("Directory created successfully");
```

```
});
```

```
// OutPut
```

```
Directory created successfully
```

If the directory already exists or there's an error (such as insufficient permissions), the `err` object will contain details.

List of fs methods in detail



4. `rmdir()`

The `fs.rmdir()` method in Node.js is used to **asynchronously remove a directory** from the file system

Syntax:- `fs.rmdir(path,[options], callback)`

```
const fs = require("fs");
```

```
// Remove an empty directory
```

```
fs.rmdir("./newDir", (err) => {
```

```
  if (err) {
```

```
    console.error("Error removing directory:", err);
```

```
    return;
```

```
  }
```

```
  console.log("Directory removed successfully");
```

```
});
```

```
// OutPut
```

```
Directory created successfully
```

If the directory does not exist or there's an error (such as insufficient permissions), the `err` object will contain details.

List of fs methods in detail

4. `isFile()`

The `isFile()` method is a function of the `fs.Stats` object in Node.js, which is used to determine if a given path refers to a file.

This method returns a boolean value indicating whether the specified path is a file or not.

Syntax:- `stats.isFile()`

```
const fs = require("fs").promises;
async function checkIfFile() {
  try {
    const stats = await fs.stat("abc.txt");
    if (stats.isFile()) {
      console.log("It is a file.");
    } else {
      console.log("It is not a file.");
    }
  } catch (err) {
    console.error("Error retrieving file stats:", err);
  }
}
checkIfFile();
```

// OutPut
It is file

List of fs methods in detail

4. createReadStream()

This methods in Node.js are used to create **readable streams** that allows you to read data from a file in chunks.

```
Syntax:-    const { createReadStream } = require('fs');  
            const stream = createReadStream(path[, options])
```

```
const fs = require("fs");  
// Create a readable stream  
const readStream = fs.createReadStream("abc.txt", { encoding: "utf8" });  
// Handle the 'data' event to read data chunks  
readStream.on("data", (chunk) => {  
    console.log("Received chunk:", chunk);  
});  
// Handle the 'end' event when the stream is finished  
readStream.on("end", () => {  
    console.log("No more data to read.");  
});  
  
// Handle errors  
readStream.on("error", (err) => {  
    console.error("Error reading file:", err);  
});
```

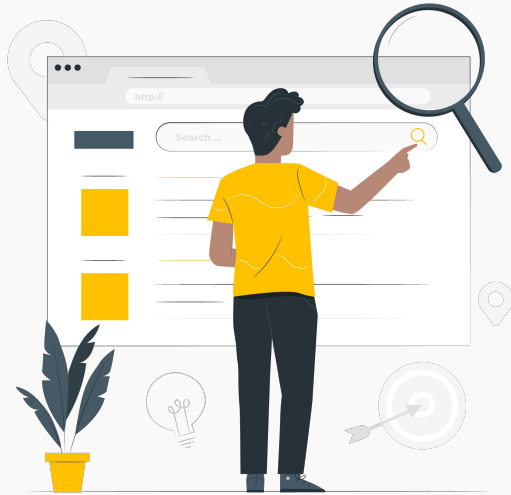
List of fs methods in detail

4. `createWriteStream()`

The `fs.createWriteStream()` is used to create a writable stream that allows you to write data to a file in chunks.

Syntax:- `const { createWriteStream } = require('fs');`
`const stream = createWriteStream(path[, options])`

```
const fs = require("fs");  
// Create a writable stream  
const writeStream = fs.createWriteStream("abc.txt");  
// Write data to the stream  
writeStream.write("Hello, World!\n");  
writeStream.write("This is a test.\n");  
// End the stream  
writeStream.end(() => {  
    console.log("Finished writing to file.");  
});  
  
// Handle errors  
writeStream.on("error", (err) => {  
    console.error("Error writing to file:", err);  
});
```

02

HTTP Module

Http module in detail

The **http module** in Node.js allows developers to create HTTP servers and clients. This server listens to incoming client requests and sends responses.

```
const http = require("http");
// Create an HTTP server
const server = http.createServer((req, res) => {
  // Set the response header and status code
  res.writeHead(200, { "Content-Type": "text/plain" });

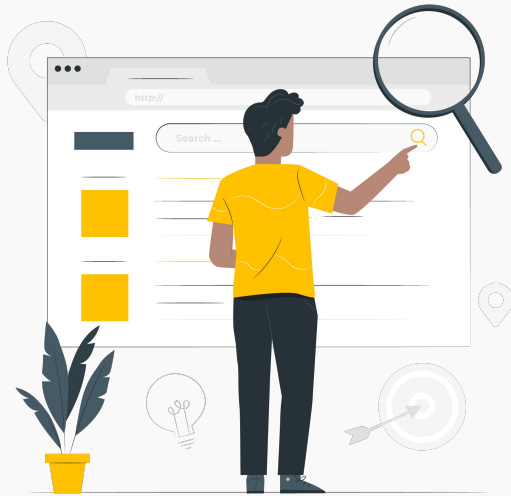
  // Send the response body
  res.end("Hello, World!\n");
});

// Server listens on port 3000
server.listen(3000, () => {
  console.log("Server running at http://localhost:3000/");
});
```

http.createServer([requestListener]):

This method creates an HTTP server object. The **requestListener** is a callback function that is executed when the server receives a request. It takes two arguments:

- **req**: Represents the incoming request object.
- **res**: Represents the response object used to send data back to the client.



03

HTTPS Module

Https module in detail



The **https module** in Node.js is similar to the **http** module but adds support for **HTTPS** (HTTP over SSL/TLS), enabling secure communication over the web.

It provides built-in methods for creating HTTPS servers and making HTTPS requests, offering encryption, data integrity, and server authentication.

Key Concepts of HTTPS in Node.js

1. **SSL/TLS:** HTTPS uses SSL (Secure Sockets Layer) or its successor TLS (Transport Layer Security) to encrypt data transferred between the client and the server, ensuring confidentiality and security.
2. **Certificates:** The server must have an SSL/TLS certificate to establish a secure connection. These certificates are typically issued by a trusted Certificate Authority (CA), but you can also use self-signed certificates for development purposes.

Https module in detail



An HTTPS server is created similarly to an HTTP server, but it requires an additional layer of encryption through SSL/TLS.

You need to provide:

- **Private key** (`key`)
- **Certificate** (`cert`)

These can either be generated for testing (self-signed certificates) or obtained from a Certificate Authority for production.

Steps to create an HTTPS server:

1. Generate SSL/TLS certificates (for development, you can generate a self-signed certificate using OpenSSL).

```
openssl req -nodes -new -x509 -keyout key.pem -out cert.pem
```

2. Use the `https.createServer()` method, providing the SSL key and certificate, and a request listener.

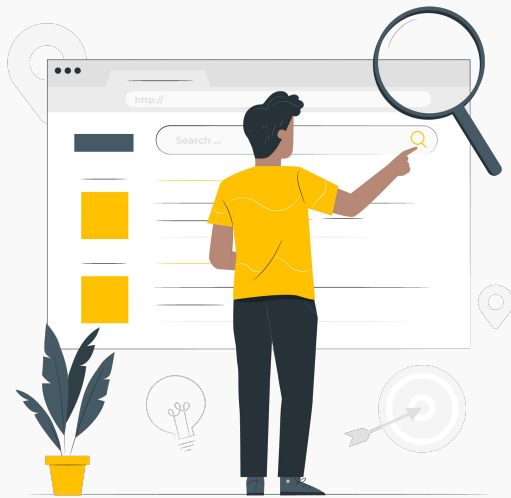
Https module in detail



```
const https = require('https');
const fs = require('fs');

// Load SSL key and certificate
const options = {
  key: fs.readFileSync('key.pem'), // Private key
  cert: fs.readFileSync('cert.pem') // Certificate
};

// Create HTTPS server
https.createServer(options, (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, Secure World!\n');
}).listen(3000, () => {
  console.log('HTTPS Server running at https://localhost:3000/');
});
```



03

Querystring Module

querystring module in detail



The **querystring** module in Node.js is used to handle query strings, typically part of URLs.

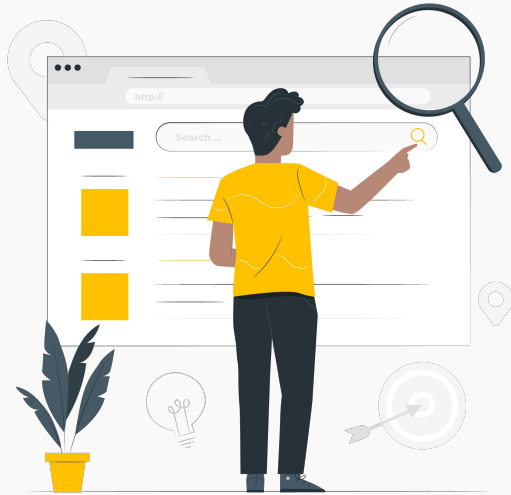
Query strings are the part of a URL that comes after the **?**, and they contain key-value pairs separated by **&**, used to pass data in web requests.

This module provides utilities for parsing and formatting URL query strings, converting them to and from objects, and handling encoding/decoding of special characters.

`stringify()` `parse()` `encode()` `decode()`

// Importing the querystring module

```
const querystring = require('querystring');  
const obj = {   name: "John Doe",   age: 30,   city: "New York"};  
  
const queryString = querystring.stringify(obj);  
console.log(queryString);  
// Output: 'name=John%20Doe&age=30&city=New%20York'
```

03

URL Module

url module in detail

The `url` module in Node.js provides utilities for parsing, constructing, normalizing, and resolving URLs.

It is particularly useful for handling URL manipulation in web applications, where you need to break down or create URLs, retrieve query parameters, or resolve relative URLs.

Key feature of URL

1. `url.parse()`: Parses a URL into its components.

```
const url = require("url");
const urlString = "https://example.com:8080/path/name?query=test#hash";
const parsedUrl = url.parse(urlString);
console.log(parsedUrl);
```

2. `url.format()`: Formats a URL object into a string.

```
const url = require("url");
const urlObject = { protocol: "https:", slashes: true,
  host: "example.com", pathname: "/path/name", search: "?query=test" };
const formattedUrl = url.format(urlObject);
console.log(formattedUrl);
// Output: 'https://example.com/path/name?query=test'
```

```
URL {
  protocol: 'https:',
  slashes: true,
  auth: null,
  host: 'example.com:8080',
  port: '8080',
  hostname: 'example.com',
  hash: '#hash',
  search: '?query=test',
  query: 'query=test',
  pathname: '/path/name',
  path: '/path/name?query=test',
  href: 'https://example.com:8080/
    path/name?query=test#hash'
}
```

url module using in http in detail

```
const http = require("http");
```

```
const url = require("url");
```

```
http
```

```
.createServer((req, res) => {
```

```
  const parsedUrl = url.parse(req.url, true);
```

```
  console.log("parse url", parsedUrl); //
```

```
  const pathname = parsedUrl.pathname;
```

```
  if (pathname === "/users/register") {
```

```
    res.writeHead(200, { "Content-Type": "text/html" });
```

```
    res.end("<h1>Register Page</h1>");
```

```
  } else {
```

```
    res.writeHead(404, { "Content-Type": "text/html" });
```

```
    res.end("<h1>404 Page Not Found</h1>");
```

```
  }
```

```
})
```

```
.listen(3000, () => {
```

```
  console.log("Server running at http://localhost:3000/");
```

```
}).
```

URL {

search: '?name=sandeep&age=23',

query: [Object: null prototype] { name:

'sandeep', age: '23' },

pathname: '/users/register',

path: '/users/register?name=sandeep&age=23',

href: '/users/register?name=sandeep&age=23'

}

Node.js

NODE PACKAGE MANAGER (npm)

- Sandeep Jaiswal (Technical lead @ Wipro)



1. Node Package manager

01

Introduction

02

Key Features

03

Package Installation



01

Introduction

Introduction



The **npm** (Node Package Manager) is a powerful tool that comes bundled with Node.js.

It is the default package manager for JavaScript and is widely used in Node.js applications to manage libraries, frameworks, and other tools.

It provides a large online registry (<https://www.npmjs.com/>) where developers can publish and discover packages.

Key Features:-

1. Package Management
2. Version Control
3. Dependency Resolution
4. Script
5. Private Package



02

Key Feature

- Package Management

Package Management using npm in detail



Npm allows developers to install, manage, and share libraries (or packages) needed for their Node.js applications.

This process is essential for organizing dependencies and maintaining project structure.

It provides a large online registry (<https://www.npmjs.com/>) where developers can publish and discover packages.

Initializing a Project

To start using npm for package management, you need to initialize your project.

This creates a package.json file that tracks your project's dependencies.

```
npm init
```

It creates file named **package.json** which is called metadata of the project.

Package Management using npm in detail



Building the **Node.js Hello World** program

Steps 1:- To verify the Node js installation using the following commands in the terminal:

```
node -v  
npm -v
```

Steps 2:- Create a new directory for your project using following command or create new folder in any drive in Windows OS:

```
mkdir my-node-project  
cd my-node-project
```

Steps 3:- Initialize the Project with **package.json**

```
npm init
```

Package Management using npm in detail

Initialize the Project with `package.json`

```
> npm init
```

Press ^C at any time to quit.

package name: (test-api) test-api

version: (1.0.0)

description: It's a initial node js application

entry point: (index.js) index.js

test command:

git repository:

keywords: node-test-api

author: Sandeep

license: (ISC)

Folder Structure:-

```
test-api
├─ package.json
```

```
// package.json
```

```
{
  "name": "test-api",
  "version": "1.0.0",
  "description": "It's a initial node js app.",
  "main": "index.js",
  "scripts": {
    "test": "echo Error: no test specified"
  },
  "keywords": [
    "node-test-api"
  ],
  "author": "Sandeep",
  "license": "ISC"
}
```

Package Management using npm in detail



Installing Project :-

To install a package locally (within your project), use the following command:

```
npm install <package-name>
```

This installs the package in the `node_modules` directory and adds it as a dependency in `package.json`.

To install a package globally (available for all projects), use the `-g` flag:

```
npm install -g <package-name>
```

This installs the `package globally`, allowing you to use it in `any Node.js` project.

Example:-

```
npm install express
```

This installs the Express framework locally.

Folder Structure:-

test-api

├─ package.json

├─ node_modules

```
// package.json
```

```
{  
  "name": "test-api",  
  "version": "1.0.0",  
  "dependencies": {  
    "express": "^4.21.1"  
  }  
}
```

Package Management using npm in detail



Manage Package Dependencies :-

The `package.json` file plays a crucial role in managing dependencies.

It has several fields relevant to package management:

- **dependencies**: Packages required for your application to run.
- **devDependencies**: Packages only needed during development (e.g., testing libraries, build tools). Install these with `--save-dev`:

```
npm install <package-name> --save-dev
```

```
// package.json
```

Using Scripts :-

You can define scripts in `package.json` to automate tasks related to your project.

Common scripts include **starting the application**, **running tests**, and **building the project**.

Run scripts using:

```
npm run <script-name>
```

eg:- `npm run dev`

```
{  
  "name": "test-api",  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js",  
    "test": "jest"  
  },  
}
```

Package Management using npm in detail



Versioning Package :-

Versioning in npm follows Semantic Versioning (SemVer), which uses the format

MAJOR.MINOR.PATCH:

- **MAJOR:** Incremented for incompatible API changes.
- **MINOR:** Incremented for adding functionality in a backward-compatible manner.
- **PATCH:** Incremented for backward-compatible bug fixes.

// package.json

```
{
  "name": "test-api",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.21.1"
  }
}
```

When installing a package, you can specify version constraints:

- **Exact Version:** `npm install <package-name>@1.2.3`
- **Minimum Version:** `npm install <package-name>@^1.2.3` (accepts any version $\geq 1.2.3$ $< 2.0.0$)
- **Latest Version:** `npm install <package-name>@latest`



03

Key Feature

- Version Control

Version Control using npm in detail



Version control in npm revolves around managing package versions in a consistent and structured way.

It ensures that projects can reliably install the correct versions of dependencies and avoid compatibility issues.

The npm adopts **Semantic Versioning**, which follows a format of **MAJOR.MINOR.PATCH** (e.g., **1.2.3**).

Common Versioning Symbols:

- **Exact version (1.2.3)**: Installs exactly this version of the package.
- **Caret (^)**: Allows updates that do not change the major version. For example, **^1.2.3** will install any version that is **>=1.2.3 <2.0.0**.
- **Tilde (~)**: Allows updates that do not change the minor version. For example, **~1.2.3** will install any version that is **>=1.2.3 <1.3.0**.
- **Greater than or equal to (>=)**: Installs the specified version or higher, for example **>=1.2.0**.
- **Less than (<)**: Installs versions lower than the specified version, for example **<2.0.0**.
- **Latest**: Installs the latest version of the package. You can specify this by using **@latest**, e.g., **npm install express@latest**.

Version Control using npm in detail



Locking Versions with `package-lock.json`.

This file ensures **exact version control** by locking the dependency tree so that the project can always install the exact same versions of every package.

Key Features of `package-lock.json`:

- **Locks exact versions:** Ensures that when the project is installed on another machine, it uses the same versions of all dependencies, even indirect dependencies.
- **Improves install times:** Speeds up `npm install` by caching exact package versions.
- **Ensures consistency:** Prevents version drift between different environments (e.g., development vs. production).

Version Management Commands

npm provides several commands for managing package versions:

Version Control using npm in detail



You can install a specific version of a package by appending `@version` to the package name:

Syntax:- `npm install <package-name>@<version>`

Example:-

```
npm install express@4.16.0
```

To update a package to the latest version, use:

Syntax:- `npm install <package-name>`

For instance, if your `package.json` lists a package as `^1.2.3`, this command will update to the latest compatible minor or patch version.

If you want to update to the latest major version, you can install it explicitly:

Syntax:- `npm install <package-name>@latest`

Version Control using npm in detail



To check if any installed packages are outdated:

```
npm outdated
```

This command will show you a list of all packages that have newer versions available than the ones currently installed.

When managing packages, you might want to address security vulnerabilities. npm provides an audit feature to help with this:

```
npm audit
```

This command checks for vulnerabilities in your dependencies. To automatically fix these vulnerabilities, you can run:

```
npm audit fix
```



04

Key Feature

- Script

Script using npm in detail

The `scripts` field in `package.json` allows you to create custom commands to run different scripts like building, testing, starting the server, linting, and more.

```
{  
  "name": "my-app",  
  "version": "1.0.0",  
  "scripts": {  
    "start": "node server.js",  
    "build": "webpack --config webpack.config.js",  
    "test": "jest",  
    "lint": "eslint src/**/*.js",  
    "dev": "nodemon server.js"  
  }  
}
```

To run a script, use the command `npm run <script-name>`. For instance:

```
npm run build
```

Script using npm in detail

You can define **pre** and **post** hooks to run scripts before or after a main script.

```
{
  "scripts": {
    "prebuild": "echo 'Running pre-build tasks ... '",
    "build": "webpack",
    "postbuild": "echo 'Build complete!'"
  }
}
```

Here:

- **prebuild** runs before **build**, echoing "Running pre-build tasks..."
- **postbuild** runs after **build**, echoing "Build complete!"

You can chain multiple commands in a single script using **&&** (runs the next command if the previous one succeeds) or **||** (runs the next command if the previous one fails).

```
{
  "scripts": {
    "test-and-lint": "npm run test && npm run lint"
  }
}
```

Script using npm in detail

Environment variables can be defined within npm scripts, allowing different configurations for each environment.

```
{
  "scripts": {
    "start": "NODE_ENV=production node server.js",
    "dev": "NODE_ENV=development nodemon server.js"
  }
}
```

In the `start` script, `NODE_ENV` is set to `production`, while in `dev`, it is set to `development`.

On Windows, you may need to install the `cross-env` package to set environment variables:

```
{
  "scripts": {
    "start": "cross-env NODE_ENV=production node server.js"
  }
}
```

Script using npm in detail

Example of a `package.json` with Advanced `npm` Scripts:-

```
{  
  "name": "my-app",  
  "version": "1.0.0",  
  "scripts": {  
    "start": "node server.js",  
    "build": "webpack --config webpack.config.js",  
    "test": "jest",  
    "lint": "eslint src/**/*.js",  
    "dev": "cross-env NODE_ENV=development nodemon server.js",  
    "prebuild": "echo 'Preparing to build ...'",  
    "prestart": "npm run build",  
    "test-and-lint": "npm run test && npm run lint",  
    "compress": "zip -r dist.zip dist/",  
    "deploy": "npm run build && npm run compress"  
  }  
}
```


Node.js

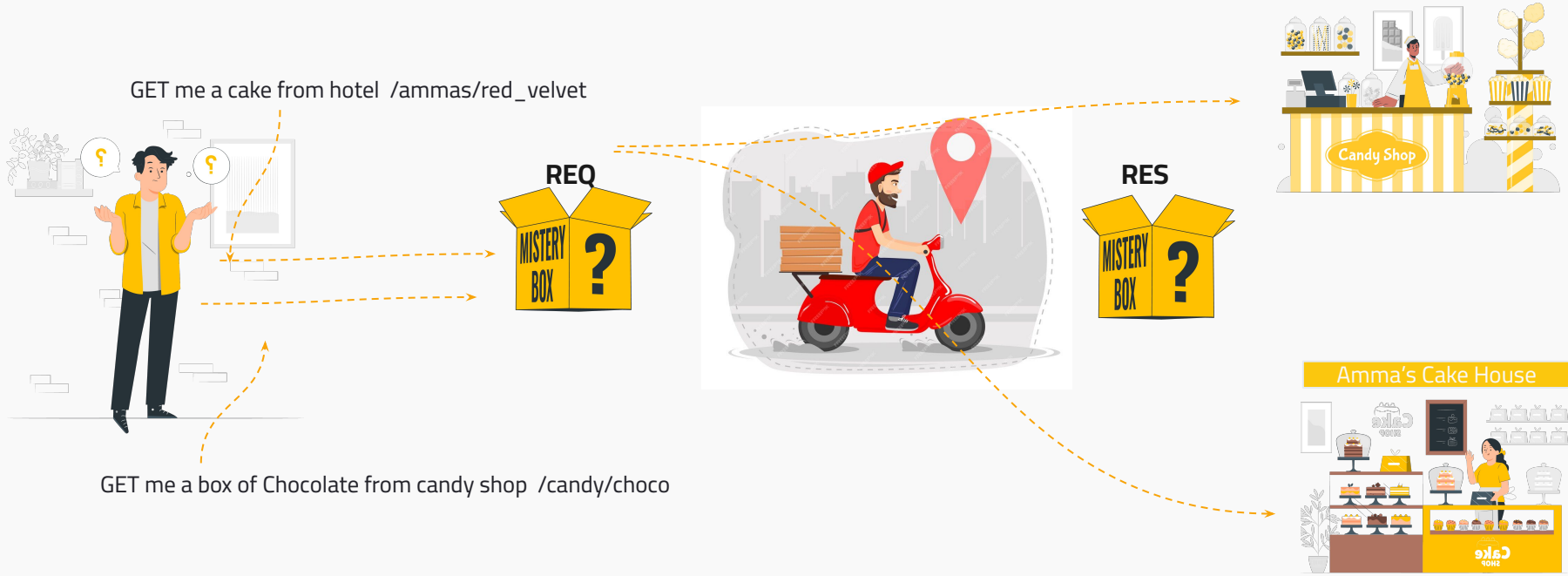
Server setup using HTTP

- Sandeep Jaiswal (Technical lead @ Wipro)



Scenario explain HTTP call Structure

`http://ammascakehouse:1st main road/cakelist/redvelvet`



Create Server using HTTP module

To create a simple HTTP server in Node.js that handles GET and POST requests, you can use Node's built-in [http](#) module.

Step-by-Step Example

1. Create a file called `server.js`.
2. Write the code below in `server.js`.

```
const http = require("http");  
// Define the port to listen on  
const PORT = 3000;  
// Create the HTTP server  
const server = http.createServer((req, res) => {  
  // Set the response header to JSON  
  res.setHeader("Content-Type", "application/json");
```

```
  // Handle GET request on the root route '/'  
  if (req.method === "GET" && req.url === "/") {  
    // Define a response object  
    const responseData = {  
      message: "Welcome to the Node.js server!",  
      info: "This is a response to a GET request.",  
    };  
    // Send the response with status 200  
    res.writeHead(200);  
    res.end(JSON.stringify(responseData));  
  }  
});  
// Start the server  
server.listen(PORT, () => {  
  console.log(`Server is running on  
http://localhost:${PORT}`);  
});
```

Node.js

Express.js Framework in detail

- Sandeep Jaiswal (Technical lead @ Wipro)



1. Express js Framework

01

API

02

Introduction

03

Server setup



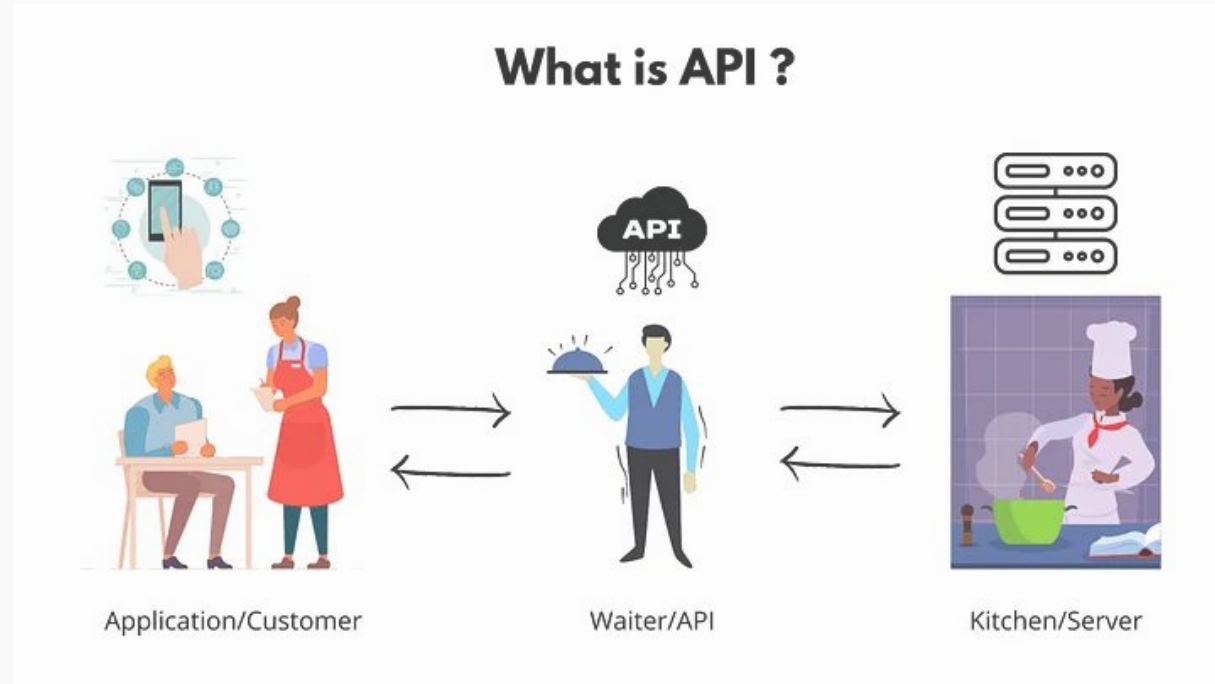
04

Application Programming Interface (API)

Application Programming Interface in detail

An **API (Application Programming Interface)** is a set of rules and protocols that allows different software applications to communicate with each other.

It acts as an intermediary between two applications, enabling them to exchange data and execute predefined actions



Application Programming Interface in detail



Components of an API

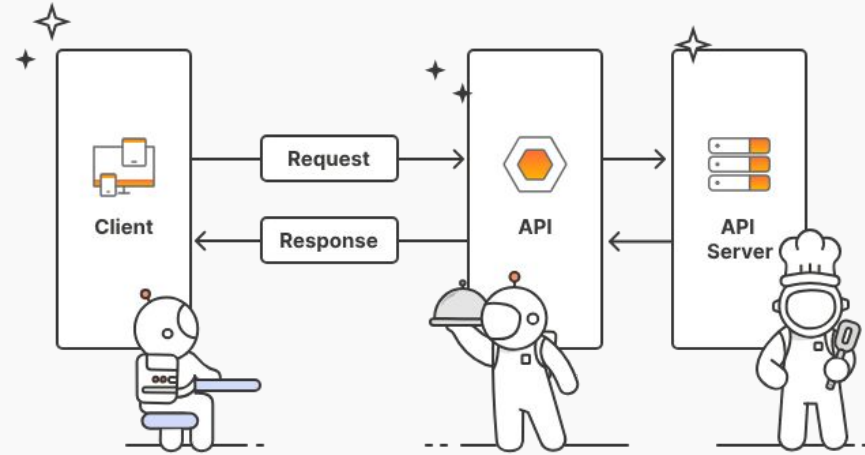
- **Request:** The request is made by a client (e.g., a web app, mobile app) to the server where the API is hosted. It typically includes:
 - **HTTP Method:** Defines the action to perform (e.g., GET, POST, PUT, DELETE).
 - **Headers:** Provide metadata for the request (like authorization, content type).
 - **Path/Endpoint:** Identifies the specific resource (e.g., `/user/123`).
 - **Parameters:** Provide additional options for the request, such as filters, search terms, or limits.
- **Response:** The server responds to the client's request with data or an acknowledgment:
 - **Status Code:** Tells the client the outcome (e.g., 200 for success, 404 for not found, 500 for server error).
 - **Body:** Contains the data, often in JSON format, if the request is successful.

Application Programming Interface in detail

Popular API Protocols

REST (Representational State Transfer):

- Most common and uses standard HTTP methods.
- Stateless and often returns data in JSON format.



SOAP (Simple Object Access Protocol):

- Uses XML for message format and can operate over various protocols.
- Provides a higher level of security (WS-Security) and transaction control, making it popular in financial and enterprise applications.

Application Programming Interface in detail



Difference between REST and SOAP API.

REST API	SOAP API
Architectural style based on HTTP.	Strict protocol with specific rules and standards.
Supports JSON, XML, HTML, and plain text (JSON is most common).	Uses only XML for message formatting.
Stateless communication, with each request being independent.	Can be stateless or stateful, allowing for more complex interactions.
Generally faster, lighter, and more efficient for web use due to lightweight data formats.	Slower due to XML's verbosity and complexity, resulting in larger payloads.
Uses standard HTTP status codes (e.g., 404 for Not Found, 500 for Server Error).	Uses specific error codes within the XML response, such as <code><faultcode></code> and <code><faultstring></code> .
Preferred for web and mobile applications due to its simplicity and flexibility.	Often used in enterprise applications where high security and transaction management are required.



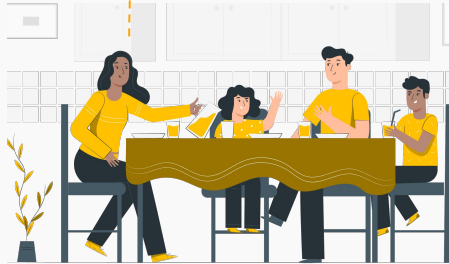
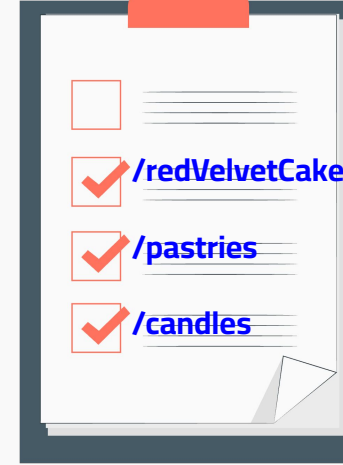
05

Introduction to Express JS

GET Request handled for server

/redVelvetCake

```
{  
  quantity: 5,  
  weight: '500 gm'  
}
```



Waiter/API
showing the
list of item



(req, res) ⇒ {
 // Prepare cake and send
}



Introduction to Express.js



Express.js is a flexible, minimalist web application framework for **Node.js**

It provides a streamlined approach to handling HTTP requests and responses, routing, middleware, and templating.

Setup Express.js Server :-

```
const express = require("express");
const app = express();
const port = 3000;
```

```
app.get("/redVelvetCake", (req, res) => {
  let reuestData = req.body;
  console.log(requestData);
  res.send("Your cake : Red Velvet Cake");
});
```

```
app.listen(port, () => {
  console.log(`Example app listening on port ${port}`);
});
```

Explaining The GET API for /redVelvetCake

/redVelvetCake

```
{  
  quantity: 5,  
  weight: '500 gm'  
}
```

Req Object

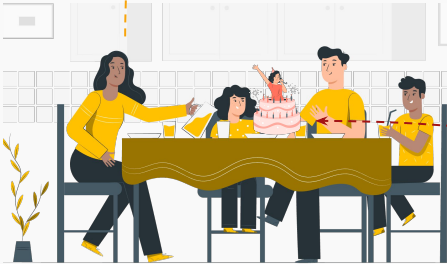
req.body -

req.url -

```
{  
  quantity: 5,  
  weight: '500 gm'  
}
```

/redVelvetCake

```
app.get("/redVelvetCake", (req, res) => {  
  let requestData = req.body;  
  console.log(requestData);  
  res.send("Your cake : Red Velvet Cake");  
});
```



Explaining The GET API for /redVelvetCake

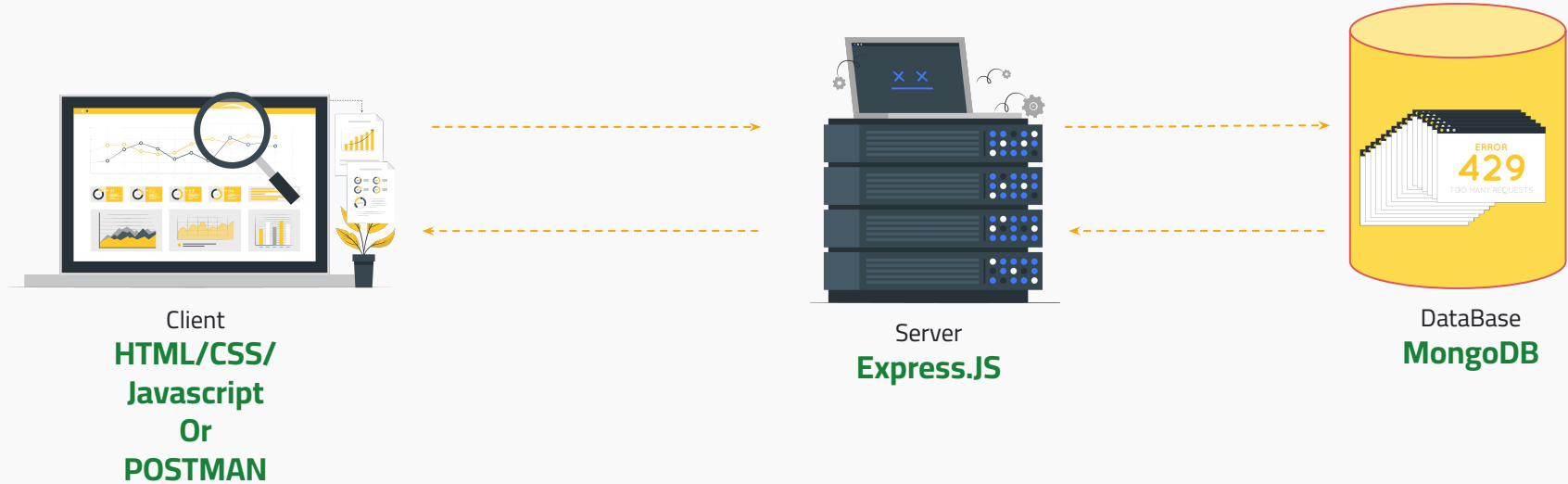
Routes Name

```
app.get("/redVelvetCake", (req, res) => {  
  let reuestData = req.body;  
  console.log(requestData);  
  res.send("Your cake : Red Velvet Cake");  
});
```

Request Data

function Handler -
Main place where cook
happening by
request and response

Connection among Systems





06

Request & Response Objects

Passing the data from client to server

`http://localhost:3000/api/cakelist/`

`http://localhost:3000/api/cakelist/:cakeType`

`http://localhost:3000/api/cakelist/:cakeType?
weight=500gm`



Client
**HTML/CSS/
Javascript
Or
POSTMAN**

```
{  
  firstName:"John",  
  lastName:"Jaiswal",  
  password:"asdf1234",  
}
```



Server
Express.JS

Request Object in detail



The `req` object represents the HTTP request and has properties for the request query string, parameters, body, HTTP headers, etc.

Key properties and methods of `req`:

<code>req.params</code>	<code>req.body</code>	<code>req.query</code>	<code>req.headers</code>	<code>req.url</code>
<code>Req.methods</code>	<code>req.path</code>	<code>req.protocol</code>	<code>req.cookie</code>	<code>req.get()</code>
<code>req.originalUrl</code>				

```
Url:- http://localhost:3000/api/cakelist/:cakeType?weight=500gm
Headers:{ 'Content-Type':'application/json'}
Body:{ 'name':'John'}
```

Request Object in detail

Any Request will have such elements :-

URL : `http://localhost:3000/api/cakelist`

Protocol : `http / https`

Query data : `http://localhost:3000/api/cakelist?weight=500gm&quantity=5`

Params data : `http://localhost:3000/api/cakelist/:cakeType`

Headers data : `headers: {
 'Authorization': 'Bearer YOUR_TOKEN',
 'Content-Type': 'application/json',
}`

Methods : `GET / POST / PUT / DELETE`

Body data:- `{
 firstName:"John",
 lastName:"Jaiswal",
 password:"asdf1234",
}`

Request Object in detail

`http://localhost:3000/api/cakelist/:cakeType?weight=500gm`

```
app.get("/api/users/cakelist/:cakeType", (req, res) => {  
  let requestQuery = req.query;  
  let requestParams = req.params;  
  let requestUrls = req.url;  
  let requestHeaders = req.headers;  
  let requestUrl = req.url;  
  let requestMethod = req.methods;  
  let requestProtocol = req.protocol;  
  console.log('Request query data: ', requestQuery);  
  console.log('Request params data: ', requestQuery);  
  console.log('Request url: ', requestUrl);  
  console.log('Request headers data: ', requestHeaders);  
  console.log('Request url data: ', requestUrl);  
  console.log('Request method data: ', requestMethod);  
  console.log('Request protocol data: ', requestProtocol);  
  res.send("Your cake : Red Velvet Cake");  
});
```

Response Object in detail

The `res` object represents the HTTP response that an Express app sends when it gets an HTTP request.

Key methods of `res`:

`res.send()` `res.json()` `res.status()` `res.set()` `res.sendFile()`

`res.send()`: Sends a response of various types (string, buffer, object, etc.).

Automatically sets the `Content-Type` header based on the data type.

```
app.get('/', (req, res) => {  
  res.send('Hello, World!');  
});
```

`res.json()`: Sends a JSON response. Automatically sets the `Content-Type` header to `application/json`.

```
app.get('/json', (req, res) => {  
  res.json({ message: 'Hello, World!' });  
});
```

Response Object in detail

res.status(): Sets the HTTP status code for the response.

```
app.get('/not-found', (req, res) => {  
  res.status(404).send('Not Found');  
});
```

res.set(): Sets response headers.

```
app.get('/', (req, res) => {  
  res.set('Content-Type', 'text/plain');  
  res.send('Hello, World!');  
});
```

res.sendFile(): Sends a file as a response.

```
app.get('/file', (req, res) => {  
  res.sendFile(path.join(__dirname,  
    'file.txt'));  
});
```

Receive request body data in detail



In Express.js, parsing request body data is crucial because it allows you to work with data sent from clients, like form submissions, JSON payloads, or file uploads.

To parse JSON data, use the `express.json()` middleware. This is useful when the client sends data in JSON format, which is typical for APIs.

```
app.use(express.json());
```

```
app.post("/api/storeCake", (req, res) => {  
  let reqBody = req.body;  
  console.log("request body ", reqBody);  
  res.send("Body data received");  
});
```

```
POST /api/cakeList HTTP/1.1  
Host: localhost:3000  
Content-Type: application/json  
Content-Length: 127  
  
{  
  "firstName": "John",  
  "lastName": "Jaiswal",  
  "phone": 2989809878,  
  "status": "INPROGRESS",  
  "active": true  
}
```


Node.js

MongoDB create DB in detail

- Sandeep Jaiswal (Technical lead @ Wipro)



Pictorial representation for Database

Products Table

name	price	description	category	createdAt
T-shirt	2000	Tshirt description	Fashion	2024-11-12
Sweatshirt	3000	Sweatshirt description	Men	2024-11-11
Sweater	1000	Sweater description	Women	2024-11-10
Shirt	4000	Shirt description	Festival	2024-11-09
Jeans	6000	Jeans description	Rough	2024-11-05

Document

```
{  
  "name": "Jeans",  
  "price": 6000,  
  "descriptions": "Jeans Description",  
  "category": "Rough",  
  "createdAt": "2024-11-05"  
}
```

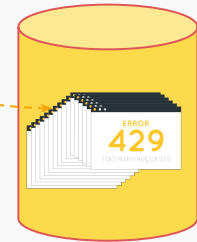
The **group of tables** are combined stored as database.

The **group of rows** stored in table.

DatabaseName ⇒ **Group of Table** ⇒ **Group of Records in row**

In MongoDB,

Tables ⇒ **Collections** **Row/Each Record** ⇒ **Documents**



MongoDB basic in detail

MongoDB is a popular, open-source, **NoSQL (Non-relational)** database that uses a flexible, document-oriented data model.

MongoDB stores data in **JSON-like documents** with dynamic schemas, which means you can store and retrieve data without having to define a strict structure in advance.

Core concepts:

- **Database:** A container for collections.
- **Collection:** A group of documents (equivalent to tables in SQL).
- **Document:** A single record (JSON-like format).

Create Database

Use command to list all the database list

```
# mongosh  
> show dbs
```

```
> show dbs  
admin                40.00 KiB  
attendance-system    212.00 KiB  
avtsl-acc-db         144.00 KiB  
config               60.00 KiB  
data-db              208.00 KiB  
db-test              72.00 KiB  
enhanced_authentication-db 144.00 KiB  
flow-proximity-db    144.00 KiB  
local                184.00 KiB
```

MongoDB Create Database



Creating a **database in MongoDB** using the **Mongo shell** is straightforward.

Instead, a database is created automatically when you insert data into it for the first time.

Before creating a database, make sure you have MongoDB installed on your system and that the MongoDB server (**mongod**) is running.

Open your terminal (command prompt) and type:

```
# mongosh
```

Before creating a new database, let's check which databases already exist on your MongoDB server.

```
> show dbs
```

```
> use products
```

```
> db :Check the current database.
```

MongoDB Create Collection

Creating a collection in **MongoDB** is a fundamental task when working with databases.

A **collection** in MongoDB is equivalent to a table in relational databases like MySQL.

It is a group of **documents** (similar to rows in tables) stored together.

Open your terminal (command prompt) and type:

```
# mongosh
```

Before creating a new database, let's check which databases already exist on your MongoDB server.

```
> show dbs
```

```
> use products
```

```
> db.createCollection(name, options);
```

MongoDB create operation to database



The **Create operation** in MongoDB allows you to **insert new documents** into a collection.

In MongoDB, this is done using methods like `insertOne()`, `insertMany()`.

In MongoDB, data is stored in **documents**, which are JSON-like objects. These documents are stored in **collections**

The `insertOne()` method is used to insert a **single document** into a collection.

Syntax:-

```
db.collection.insertOne(document)
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId("64fc2f4f9abf7c8b7c12d567")
}
```

```
db.users.insertOne({
  name: "John Doe",
  age: 30,
  email: "john@example.com",
  hobbies: ["reading", "traveling"],
  address: {
    city: "San Francisco",
    state: "CA"
  }
})
```

MongoDB bulk create operation to database



The `insertMany()` method is used to insert **multiple documents** into a collection at once.

Syntax :-

```
db.collection.insertMany([document1, document2, ... ])
```

Examples:-

```
db.products.insertMany([
  { name: "Laptop", price: 1200, category: "Electronics" },
  { name: "Headphones", price: 150, category: "Accessories" }
])
```

Output:-

```
{
  acknowledged: true,
  insertedIds: {
    "0": ObjectId("64fc2f8d9abf7c8b7c12d570"),
    "1": ObjectId("64fc2f8d9abf7c8b7c12d571")
  }
}
```

MongoDB read operation to database



The **Read operation** in MongoDB involves **retrieving documents** from a collection

In MongoDB, this is done using methods like `find()` and `findOne()` methods.

MongoDB offers a rich set of query capabilities that allow you to filter, sort, and project data to get the exact information you need.

The `find()` method retrieves **multiple documents** from a collection that match a specified query.

The `findOne()` method retrieves a **single document** that matches the query.

Syntax:-

```
db.collection.find(query, projection)
```

- **query:** Specifies the criteria for selecting documents (optional).
If omitted, all documents are returned.
- **projection:** Specifies which fields to include or exclude (optional).

```
db.users.find({ city: "New York" })
```

```
db.users.findOne({ name: "Alice" })
```

```
db.users.find({}, { _id: 0 })
```

Here above `_id` is ignored in response since `_id:0`

Node.js

Mongodb CRUD(read/update) operation

- Sandeep Jaiswal (Technical lead @ Wipro)



MongoDB read operation Comparison operator



MongoDB supports various comparison operators like `$gt`, `$gte`, `$lt`, `$lte`, `$ne`, and `$in`

1. Find documents where `age` is greater than `25`

Example:-

```
db.myCollection.find({ age: { $gt: 25 } })
```

2. Find documents where `status` is either `"active"` or `"inactive"`:

```
db.myCollection.find({ status: { $in: ["active", "inactive"] } })
```

3. To find documents where `age` is greater than `25` and `status` is `"active"`:

```
db.myCollection.find({ age: { $gt: 25 }, status: "active" })
```

MongoDB read operation Logical operator



MongoDB supports logical operators like `$or`, `$and`, `$not`, and `$nor`.

1. Find documents where `age` is less than `20` or `status` is "inactive":

```
db.users.find({
  $or: [{ age: { $lt: 20 } }, { status: "inactive" }]
})
```

2. Find documents where `age` is greater than `25` **AND** `status` is "active":

```
db.users.find({
  $and: [{ age: { $gt: 25 } }, { status: "active" }]
})
```

3. If your document has nested fields, use dot notation. For example, to find documents where the `address.city` is "Mumbai":

```
db.myCollection.find({ "address.city": "Mumbai" })
```

MongoDB limit, skip, sort method

Mongodb support method for **limiting** , **skipping** and **sorting** the output documents

1. **Limit** the number of results:

```
db.users.find().limit(5)
```

2. **Sort** results by **age** in ascending order (1 for ascending, -1 for descending):

```
db.users.find().sort({ age: 1 })
```

3. **Skip** the first 3 documents:

```
db.myCollection.find().skip(3)
```

MongoDB Update operation in detail



In MongoDB, the **update operations** are used to modify documents in a collection.

MongoDB provides several methods to update documents, including updating specific fields, replacing entire documents, and performing bulk updates.

MongoDB Update Methods

1. **updateOne()**: Updates a single document that matches the specified filter.
2. **updateMany()**: Updates all documents that match the specified filter.
3. **replaceOne()**: Replaces a single document with a new document.

The basic structure of an update command:

```
db.collection.updateOne(  
  { filter },           // The condition to match documents  
  { update operations }, // The update operation(s)  
  { options }           // Optional: settings like upsert, etc  
);
```

MongoDB Update operation in detail



The `updateOne()` method updates a **single document** that matches the filter.

Supports using **update operators** like `$set`, `$unset`, `$inc`, etc.

Syntax:-

```
db.collection.updateOne(filter, update, options)
```

Update the `status` field of the first document where `name` is "Sandeep":

```
db.users.updateOne({ name: "Sandeep" }, { $set: { status: "active" } });
```

The `updateMany()` method updates **all documents** that match the filter. Useful for bulk updates.

Syntax:-

```
db.collection.updateMany(filter, update, options)
```

MongoDB Update operation in detail

Increase the **age** by 5 for all users with **status: "inactive"**:

```
db.users.updateMany({ status: "inactive" }, { $inc: { age: 5 } });
```

Remove a field from all documents:

```
db.users.updateMany({}, { $unset: { temporaryField: "" } });
```

The **replaceOne()** method replaces the **entire document** that matches the filter with a new document.

The document structure will be completely replaced, except for the **_id** field.

```
db.users.replaceOne(  
  { name: "Sandeep" },  
  { name: "Sandeep", age: 30, status: "active", department: "Engineering" }  
);
```

MongoDB Update operators in detail

let's review some important **update operators**:

<u>Operator</u>	<u>Description</u>
<code>\$set</code> ----->	Sets the value of a field. Adds the field if it does not exist.
<code>\$unset</code> ----->	Removes the specified field from a document.
<code>\$inc</code> ----->	Increments the value of a field by a specified amount.
<code>\$mul</code> ----->	Multiplies the value of a field by a specified factor.
<code>\$rename</code> ----->	Renames a field.
<code>\$min</code> ----->	Updates the field if the specified value is less than the current value.
<code>\$max</code> ----->	Updates the field if the specified value is greater than the current value.
<code>\$push</code> ----->	Appends a value to an array.
<code>\$pop</code> ----->	Removes the first or last element from an array.
<code>\$addToSet</code> ----->	Adds a value to an array only if it doesn't already exist.
<code>\$pull</code> ----->	Removes all instances of a value from an array.

MongoDB Update operators example

`$inc` -----> Increments the value of a field by a specified amount.

Increase the `age` by 2 for all users whose `status` is "inactive":

```
db.users.updateMany({ status: "inactive" }, { $inc: { age: 2 } });
```

`$rename` -----> Renames a field.

Rename the `nickname` field to `alias` for all documents:

```
db.users.updateMany({}, { $rename: { nickname: "alias" } });
```

`$push` -----> Appends a value to an array.

Add an item to an array:

```
db.users.updateOne({ name: "Sandeep" }, { $push: { hobbies: "coding" } });
```

MongoDB Update operators example



`$addToSet` -----> Adds a value to an array only if it doesn't already exist.

Add "guitar" to the `hobbies` array only if it doesn't already exist:

```
db.users.updateOne({ name: "Sandeep" }, { $addToSet: { hobbies: "guitar" } });
```

`$pull` -----> Removes all instances of a value from an array.

Remove an item from an array:

```
db.users.updateOne({ name: "Sandeep" }, { $pull: { hobbies: "coding" } });
```

`$min` -----> Updates the field if the specified value is less than the current value.

Let's update the student's score to **75** if the current score is **greater than 75**.

```
db.students.updateOne( { _id: 1 }, { $min: { score: 75 } });
```

MongoDB Delete operation with example



In MongoDB, the **delete operations** are used to remove documents from a collection.

Types of Delete Operations

1. **deleteOne()**: Deletes a single document that matches the specified filter.
2. **deleteMany()**: Deletes all documents that match the specified filter.
3. **findOneAndDelete()**: Finds and deletes a single document, returning the deleted document.

The **deleteOne()** method deletes **one document** that matches the filter criteria. If multiple documents match, only the **first document** found will be deleted.

Syntax:-

Example:-

```
db.collection.deleteOne(filter)
```

```
db.users.deleteOne({ name: "Sandeep" });
```

MongoDB Delete operation with example



The `deleteMany()` method deletes **all documents** that match the filter criteria.

Useful for bulk deletion when you need to remove multiple documents based on a certain condition.

Syntax:-

```
db.collection.deleteMany(filter)
```

Example:- Delete all users who are `inactive`:

```
db.users.deleteMany({ status: "inactive" });
```

The `findOneAndDelete()` method finds a single document, deletes it, and **returns** the deleted document.

It is useful when you need to remove a document and use its data for further processing.

Syntax:-

Example: Find and delete a user with `name: "Sandeep"`, and return the deleted document:

```
db.collection.findOneAndDelete(filter, options)    db.users.findOneAndDelete({ name: "Sandeep" });
```

Options:

- `sort`: Sorts the documents before deleting, so you can control which document gets deleted if multiple documents match.
- `projection`: Specifies the fields to return in the deleted document.

MongoDB Delete operation with example



How to Delete All Documents in a Collection

If you want to delete **all documents** in a collection but keep the collection itself, use an empty filter with `deleteMany()`:

```
db.users.deleteMany({});
```

How to Drop a Collection

If you want to delete the entire collection (including its indexes), use:

```
db.users.drop();
```

How to Drop a Database

To delete an entire database:

```
db.dropDatabase();
```